

Token and cost statistics: Cost observability becomes even more critical as organisations scale AI deployments. For LLM apps, teams should constantly monitor:

- Input tokens
- Output tokens
- Total token consumption
- Cost per request
- Cost per user
- Daily and monthly inference costs

Tracking these metrics allows organisations to optimize prompt design, select the right models and manage operational costs.

Retrieval metrics for RAG systems:

In RAG architectures, the quality of the response is directly affected by the quality of the retrieved information. This includes:

- Retrieval latency
- Top-k retrieval accuracy
- Context relevance
- Document freshness
- Vector database query performance
- Retrieval success rate

These indicators allow teams to determine if the retrieval layer is

delivering relevant and useful context to the language model.

Quality and user experience

measures: In many cases, the most useful signals concern the quality of the generated responses. Organisations should pay attention to:

- Response relevance
- Hallucination rate
- Faithfulness to source documents
- User satisfaction scores
- User feedback
- Resolution rate
- Task completion rate

These metrics provide insight into whether the AI system is generating meaningful business value and meeting user expectations.

Organisations can gain a holistic view of system health and performance by combining infrastructure, application, AI workload, cost, retrieval and quality metrics. This holistic approach is the foundation of effective AI observability.

Reference architecture for AI observability

A production AI application typically

consists of several interconnected components including orchestration frameworks, vector databases, language models, external APIs, and supporting infrastructure. Observability must therefore span the entire request lifecycle rather than focusing on a single component.

When a user submits a query, the request passes through the application layer and orchestration framework, where prompts are constructed and relevant context is retrieved. The retrieval layer may interact with vector databases, enterprise knowledge sources, or external services before sending the enriched prompt to the language model.

At each stage, telemetry data should be captured and analysed. OpenTelemetry can be used to generate distributed traces, allowing teams to follow requests across multiple services. Prometheus collects operational metrics such as latency, throughput, resource utilisation, and error rates, while Grafana provides dashboards and alerting capabilities.

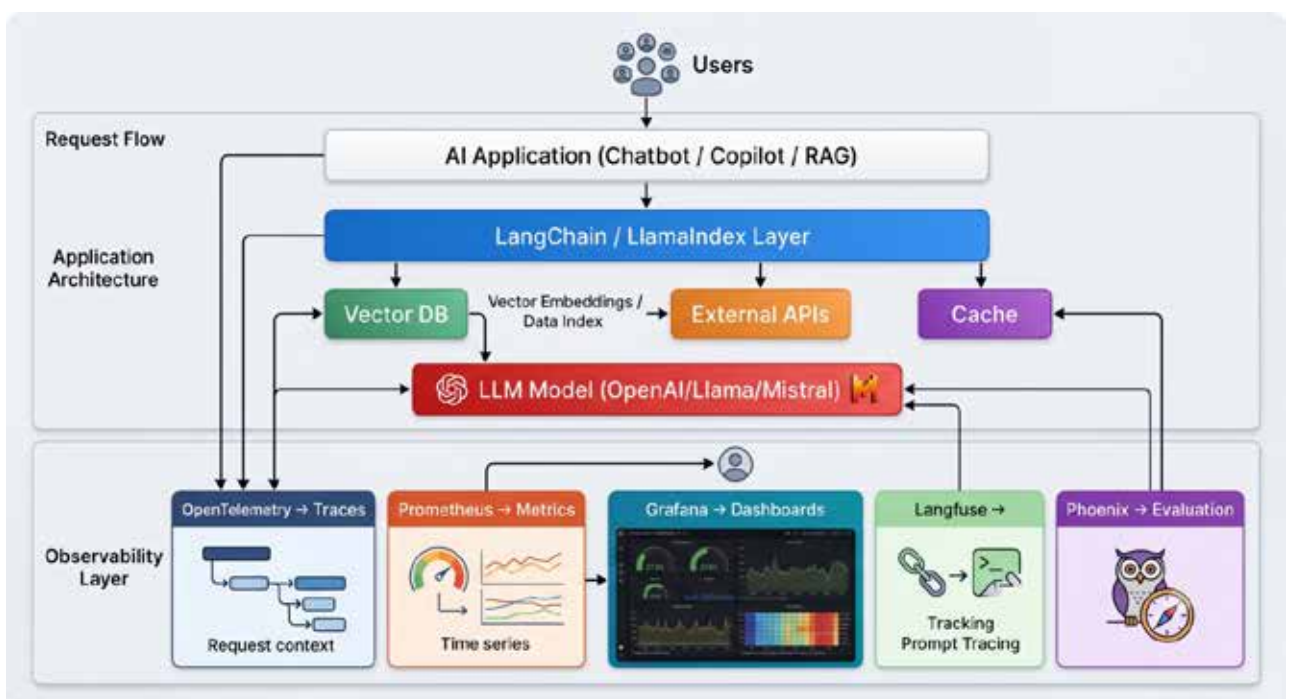


Figure 4: Reference architecture for observing AI workloads and LLM applications